TP Kubernetes

Premiers déploiements avec Minikube et Docker

Élément	Détail
Binôme	Cheick COULIBALY & Yoann DOSSOU-YOVO
Formation	Master 1 Data Engineering & IA EFREI Paris
Date	19 octobre 2026

1. Environnement et versions

Outil	Version
Docker	29.8.1
kubectl	v1.36.1
Minikube	v1.39.0
Kubernetes (nœud)	v1.37.0

2. Démarrage et vérification du cluster

Nœud identifié : minikube (statut Ready, rôle control-plane, IP interne 192.168.49.2).

Composants système observés (namespace kube-system) :

- **kube-apiserver** : point d'entrée du cluster, reçoit toutes les commandes kubectl.
- **etcd** : base de données qui stocke l'état du cluster.
- **kube-scheduler** : choisit sur quel nœud placer chaque Pod.
- Autres : kube-controller-manager, coredns, kube-proxy, kindnet, storage-provisioner.

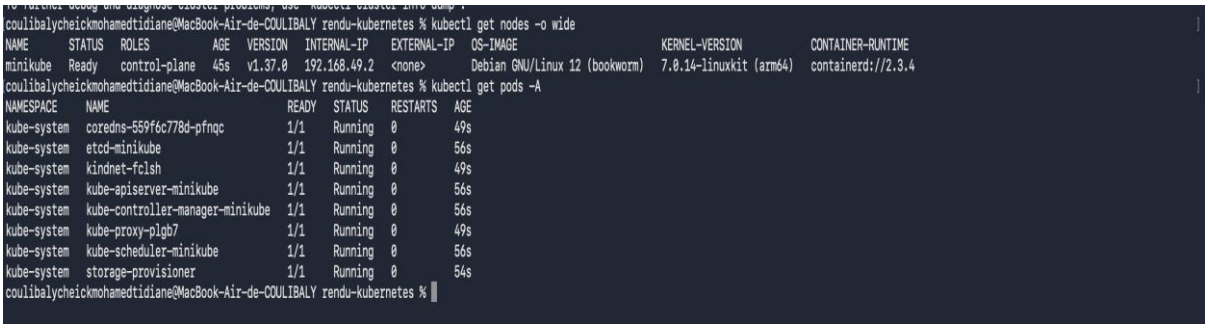


Figure 1 : Nœud minikube « Ready » et Pods système en cours d'exécution

3. Namespace et Pod autonome

Création d'un namespace demo, défini comme namespace par défaut :

```
kubectl create namespace demo
kubectl config set-context --current --namespace=demo
```

Manifeste manifests/pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: whoami-pod
  labels:
    app: whoami-pod
spec:
  containers:
    - name: whoami
      image: traefik/whoami:v1.10.3
      ports:
        - containerPort: 80
```

```
kubectl apply -f manifests/pod.yaml
kubectl get pods -o wide
kubectl describe pod whoami-pod
kubectl logs whoami-pod
kubectl delete pod whoami-pod
kubectl get pods
```

Après suppression, kubectl get pods renvoie « No resources found in demo namespace » : le Pod autonome ne revient pas, car aucun contrôleur ne le surveille.

4. Deployment à trois réplicas

Manifeste manifests/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: whoami
spec:
  replicas: 3
  selector:
    matchLabels:
      app: whoami
  template:
    metadata:
      labels:
        app: whoami
    spec:
      containers:
        - name: whoami
          image: traefik/whoami:v1.10.3
          ports:
            - name: http
              containerPort: 80
```

Le sélecteur (spec.selector.matchLabels) et les labels du modèle de Pod (spec.template.metadata.labels) portent la même valeur app: whoami. Le port est nommé http pour être référencé par le Service.

```
kubectl apply -f manifests/deployment.yaml
kubectl get deployment,replicaset,pods
kubectl get pods --show-labels
kubectl describe deployment whoami
```

Résultat : le Deployment whoami est à 3/3, le ReplicaSet whoami-c6f799d48 gère les 3 Pods, tous en Running avec le label app=whoami.

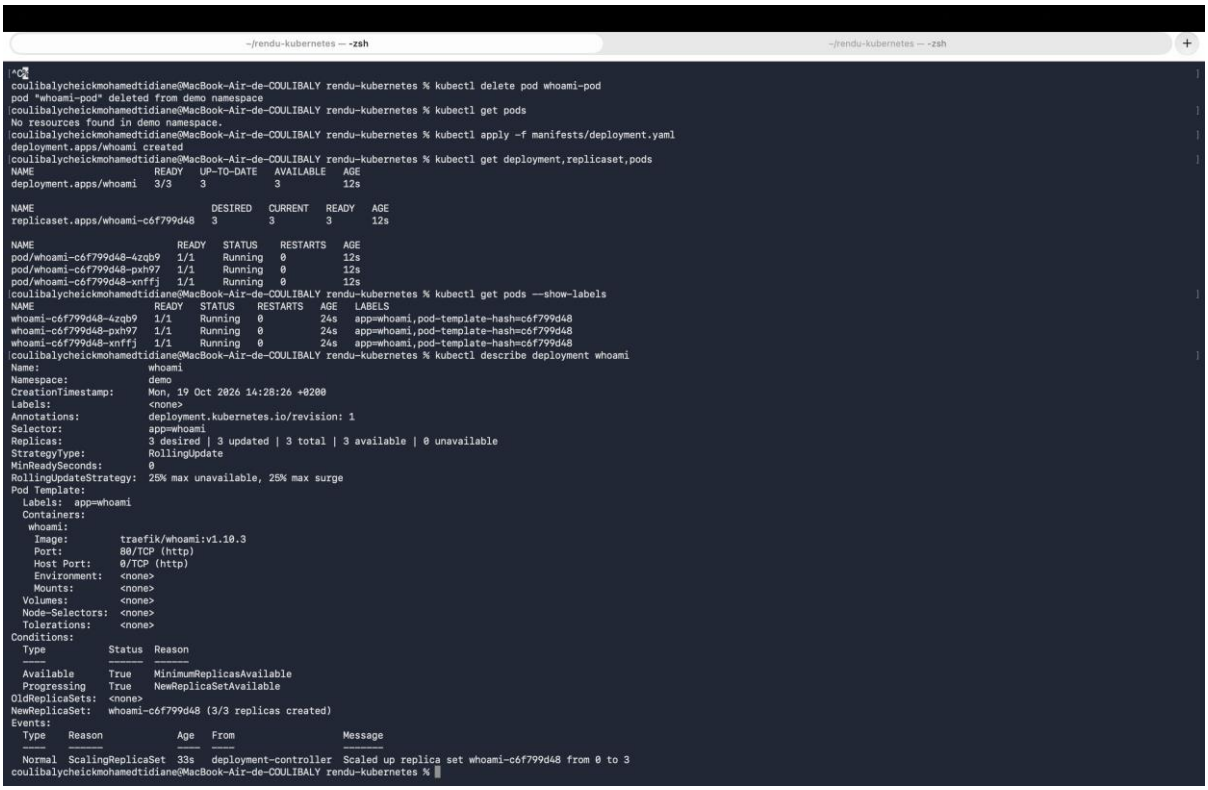


Figure 2 — Deployment, ReplicaSet et 3 Pods whoami, labels et description du Deployment

Auto-réparation

Un terminal observe les Pods en direct (kubectl get pods -w) pendant qu'un second supprime le Pod whoami-c6f799d48-4zqb9.

Observation : le Pod supprimé passe en Terminating, et un nouveau Pod (whoami-c6f799d48-jqjwg) est immédiatement créé (Pending → ContainerCreating → Running). On retrouve ensuite 3 Pods.

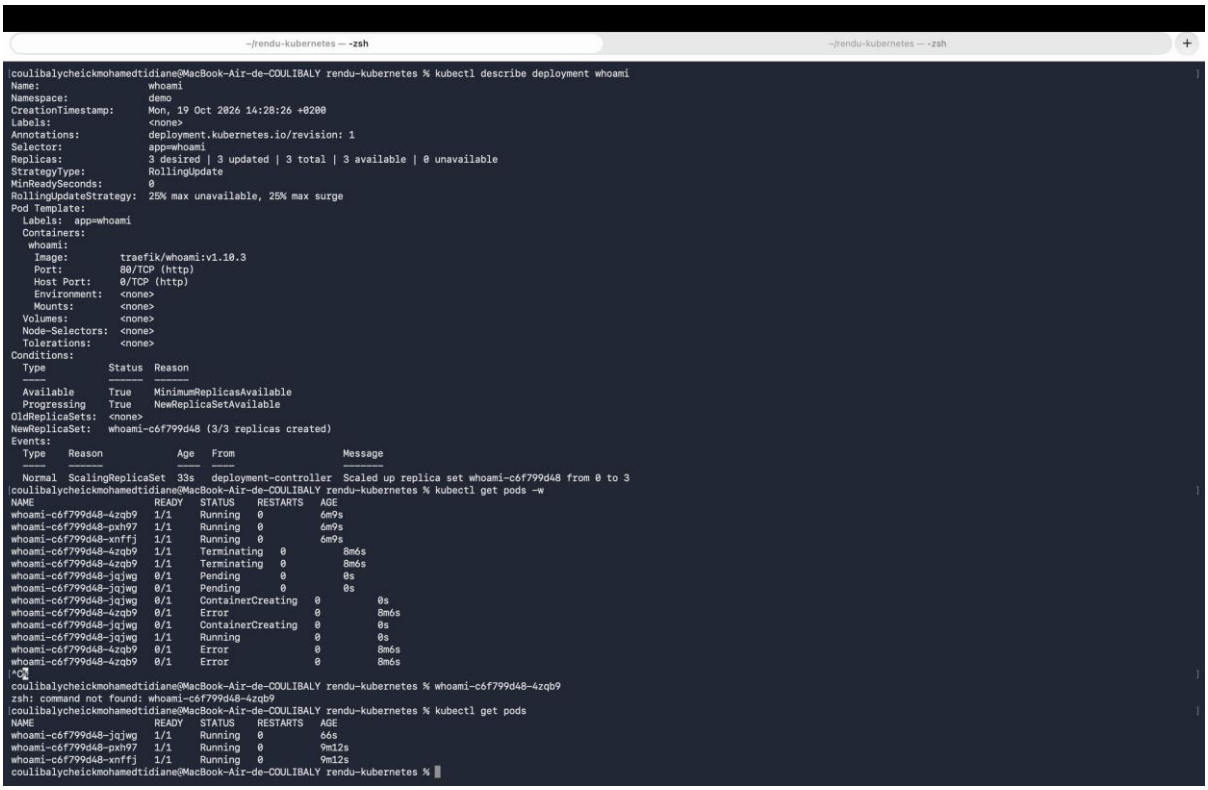


Figure 3 — Suppression d'un Pod et recréation automatique par le ReplicaSet

Changement d'échelle

```
kubectl scale deployment whoami --replicas=5
kubectl scale deployment whoami --replicas=0
kubectl scale deployment whoami --replicas=3
kubectl rollout status deployment/whoami
```

Le nombre de Pods suit la valeur demandée : 5, puis 0, puis retour à 3.

5. Exposition avec un Service

Manifeste manifests/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: whoami-service
spec:
  type: ClusterIP
  selector:
    app: whoami
  ports:
    - port: 80
      targetPort: http

kubectl apply -f manifests/service.yaml
kubectl get services
kubectl get endpoints whoami-service
kubectl port-forward service/whoami-service 8080:80
curl http://localhost:8080
```

L'application répond via le Service. Le champ Hostname indique quel Pod a traité la requête ; il change selon les requêtes, ce qui montre que le Service répartit le trafic entre les Pods.

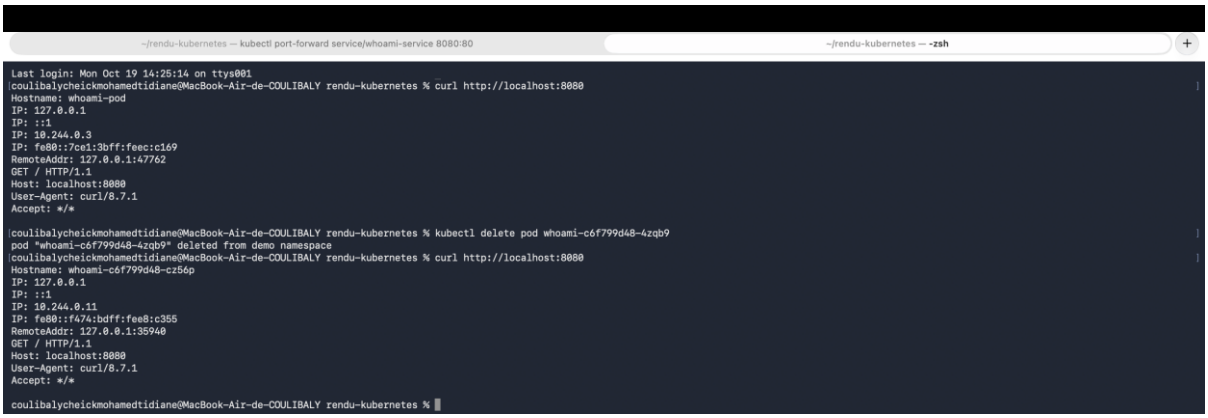


Figure 4 — Réponse HTTP de whoami via le Service (port-forward 8080 → 80)

6. Image Docker personnalisée

Une page monapp/index.html (titre « Bonjour depuis BABI ! » et un court message) est servie par nginx.

monapp/Dockerfile

```
FROM nginx:1.27
COPY index.html /usr/share/nginx/html/index.html

docker build -t monapp:1.0 ./monapp
minikube image load monapp:1.0
```

Manifeste monapp-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: monapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: monapp
  template:
    metadata:
      labels:
        app: monapp
    spec:
      containers:
        - name: monapp
          image: monapp:1.0
          imagePullPolicy: Never
          ports:
            - containerPort: 80

kubectl apply -f monapp-deployment.yaml
kubectl rollout status deployment/monapp
kubectl get pods -l app=monapp
kubectl port-forward deployment/monapp 8081:80
```

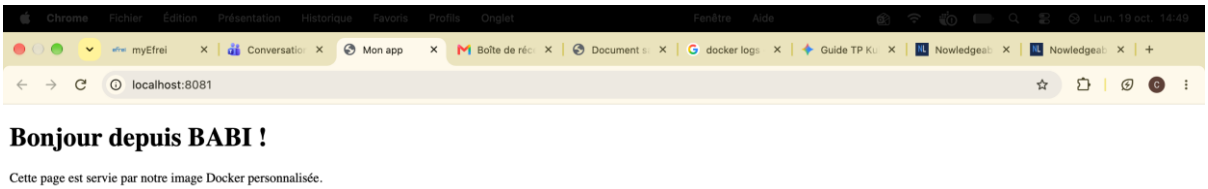


Figure 5 — Page de l'image monapp:1.0 affichée sur http://localhost:8081

7. Questions de synthèse

Pod autonome vs Pod géré par un Deployment

Un Pod autonome n'est surveillé par personne : s'il est supprimé ou plante, il disparaît. Un Pod géré par un Deployment est recréé automatiquement, et on peut changer facilement le nombre de copies ou la version de l'image.

Rôle du ReplicaSet dans l'auto-réparation

Le ReplicaSet compare en continu le nombre de Pods voulu (3) au nombre réel. S'il en manque un, il en crée un nouveau. C'est ce qu'on a vu en figure 3.

Rôle du Service

Le Service donne une adresse stable pour joindre des Pods dont les IP changent, et répartit le trafic entre eux. Il choisit ses Pods grâce aux labels (app: whoami). On vérifie qu'il cible les bons Pods avec kubectl get endpoints whoami-service, qui doit lister les IP des Pods whoami, et en testant avec curl.

Pourquoi minikube image load et imagePullPolicy: Never

L'image monapp:1.0 est construite sur notre machine et n'existe sur aucun registre (Docker Hub). Le cluster Minikube a son propre stockage d'images : minikube image load y copie l'image. imagePullPolicy: Never interdit à Kubernetes d'essayer de la télécharger, ce qui provoquerait une erreur ErrImagePull.

8. Difficultés rencontrées

- **Shell absent dans l'image whoami :** kubectl exec -it whoami-pod -- sh ne fonctionne pas car l'image est minimaliste. Le hostname a été vérifié via la réponse HTTP (champ Hostname).
- **Erreur de saisie :** le nom du Pod a d'abord été tapé seul (« command not found ») ; la bonne commande est kubectl delete pod <nom>.
- **Ciblage par labels :** lors d'un test, une réponse venait de whoami-pod. Un Service envoie le trafic à tout Pod portant le label sélectionné ; il faut donc donner des labels distincts au Pod autonome et au Deployment.

9. Nettoyage

```
kubectl delete -f manifests/service.yaml --ignore-not-found
kubectl delete -f manifests/deployment.yaml --ignore-not-found
kubectl delete -f manifests/pod.yaml --ignore-not-found
kubectl delete -f monapp-deployment.yaml --ignore-not-found
kubectl delete namespace demo
kubectl config set-context --current --namespace=default
minikube stop
```